6CM504 Concurrency and communication

Modelling and verification

Prof. Alistair A. McEwan

School of Computing
University of Derby

Table of Contents

This week

We have now looked at specification and refinement: two very important concepts that allow us to consider what our systems do. We have looked at both recursive, and non-recursive processes—including special cases such as the deadlocked process `STOP`. However we have not specified what we mean by *the deadlocked process*. This is a fundamental concept to our models and so in this lecture we shall explore topics related to process termination—when processes may, or may not have successfully completed. We will develop this into one of the most fundamental concepts in Engineering—the ability to build systems (in our case, models) in a compositional and modular fashion.

Table of Contents

What is termination?

1. In many programming languages there is a sequential composition operator ;
2. The structure `P; Q` would normally be expected to run `P` until it terminates, and then to run `Q` until it terminates
3. In CSP, this means we would expect to see all of `P`'s communication until it terminates, and then the same for `Q`
4. *Termination* is an important property to understand rigorously—there is no conceptual problem with this provided we understand exactly what it means

Processes that cease to communicate

1. Previously, we have seen the process `STOP` which is an example of a *deadlocked process* as it does nothing
2. We also mentioned the process `div` which we will say more about later (but for now, looks very much like `STOP`)
3. Neither of these two processes can in fact be said to have terminated correctly—they represent error states

Why do we need to understand termination?

1. The first question is, why do we need to model termination?
2. Consider our example `P; Q`
3. It is clear that there is a point where control should be relinquished by `P` and taken up by `Q`
4. If we are able to argue about the behaviours of `P` and `Q`, we need to be able to model them with a form of termination that happens positively rather than by default

Table of Contents

SKIP

1. Now we meet a new process one which terminates immediately and does nothing

   `SKIP`

2. We think of the act of termination as producing the special event ✓ (pronounced "tick")
3. `SKIP` can therefore be defined as the process $\checkmark \rightarrow STOP$
4. This process has the traces $\{\langle\rangle, \langle\checkmark\rangle\}$
5. We think of communicating ✓ as the process saying "I have terminated successfully"

Some ✓ details

1. The identification of termination with communicating an event (✓) is convenient and helps us with modelling
2. Care needs to be taken as it is treated specially
   - ✓ is always the final event a process performs
   - $\checkmark \notin \Sigma$—emphasising that it is a special event
   - We never introduce ✓ directly—we only use `SKIP`
   - So we would never really have written $\checkmark \rightarrow STOP$ in a process
3. This has theoretical implications, but we will not focus on these—we will focus on what this means in practice for our models

Termination and sequential composition

1. The ✓ event is handled specially for sequential composition
2. Consider the processes `a -> SKIP` and `b -> SKIP`
3. They have traces $\{\langle\rangle, \langle a\rangle, \langle a, \checkmark\rangle\}$ and $\{\langle\rangle, \langle b\rangle, \langle b, \checkmark\rangle\}$
4. Now consider their sequential composition `a -> SKIP; b -> SKIP`
5. This has traces $\{\langle\rangle, \langle a\rangle, \langle a, b\rangle, \langle a, b, \checkmark\rangle\}$

Some interesting compositions

1. This gives rise to some new unit laws

```
P; SKIP = P     (;-unit-right)

SKIP; P = P     (;-unit-left)
```

Termination and deadlock

1. Question: what can you say about the following processes and their traces, assuming

```
P = a -> SKIP
Q = b -> SKIP
R = c -> STOP
```

   1. `P; Q`
   2. `P; R`
   3. `R; P`

Termination and deadlock

1. Question: what can you say about the following processes and their traces, assuming

```
P = a -> SKIP
Q = b -> SKIP
R = c -> STOP
```

   1. `P; Q`
   2. `P; R`
   3. `R; P`

2. Conclusion: we cannot differentiate between successful termination and deadlock!

Finding deadlocks

1. The ability to differentiate between termination and deadlock is hugely important to us
2. Generally we consider deadlock to be a bad thing as there is no progress begin made
3. If we consider a process `P` with some alphabet `X`, the process `STOP` will mean that ✓ will never appear in the trace
4. We can test this using trace refinement checks in FDR

Table of Contents

What is hiding?

1. In general the alphabet of a process contains just those events which are considered to be relevant (and whose occurrence requires simultaneous participation of the environment)
2. However in describing the internal behaviour of a machine we often need to consider events representing internal transitions of that mechanism
3. When we have described that mechanism we may wish to conceal its workings from the outside world
4. This is a fundamental concept to Engineering—the ability to build systems in a compositional fashion, taking a black box view of components

Hiding

1. After constructing a process we may wish to conceal the structure of its components or internal workings (including actions internal to the mechanism)
2. In fact when we conceal internal components, we imagine them occurring automatically and instantly as soon as they can, without the environment controlling them
3. If `C` is a (finite) set of events to be made internal to a process (concealed) in this was, then the process

   ```
   P \ C
   ```

   is a process that behaves exactly as `P` but with the occurrence of all events in the set `C` concealed

Hiding: an example

1. The manufacturer of our vending machine has used cheap components—and these components are noisy!

```
NoisyVM = {coin, choc, clink, clunk, toffee}
```

where `clink` is the sound of a coin dropping into the money box of a noisy vending machine, and `clunk` is the sound made by the vending machine on completion of a transaction

A noisy vending machine!

1. The cheap and nasty noisy vending machine looks like this

```
NoisyVM =
  coin ->
    clink -> (
      choc -> clunk -> NoisyVM
      []
      toffee -> clunk -> NoisyVM )
```

2. Question: how will the traces of our noisy vending machine differ from the simple `OptionVM` we saw in previous weeks?

Silencing the vending machine

1. The people working in the office next to the vending machine have asked that it is placed in a soundproof box, so they are not disturbed by constant clinking and clunking
2. This is easy enough

```
SilencedVM = NoisyVM \ {clink, clunk}
```

3. Now no-one will observe either a `clink` or a `clunk`, unless they look inside the `NoisyVM`
4. Question: how do the traces of our `SilencedVM` differ from those of the `OptionVM` we saw in previous weeks?

Some laws of hiding

1. There are a number of laws of hiding that help us in reasoning about processes
   - Hiding nothing leaves a process unchanged

```
P \ {} = P
```

   - Hiding one set of symbols, and then some more, is the same as hiding them all

```
(P\B)\C = P\(union(B,C))
```

   - Hiding distributes through internal choice

```
(P |~| Q)\C = (P\C) |~| (Q\C)
```

   - Hiding does not change the behaviour of the stopped process, only its alphabet

```
STOP(A)\C = STOP(A\C)
```

Step laws

1. Step laws are a family of laws that describe how we can "step through" a process, one observation at a time
2. In this module we have not considered step laws for the operators we have seen
3. However hiding has a very important step law that we cannot ignore
4. The purpose of hiding is to allow any of the hidden events to occur automatically and instantly, but make such occurrences invisible. Unconcealed events remain unchanged

```
(x -> P) \ C = x -> (P\C)      if not(member(x, C
             = P\C             if member(x, C)
```

A point to ponder

1. Here is a challenge for the class
2. There will be a prize for person offering the first correct answer (assuming it isn't me)
3. We saw a step law describing how hiding distributes though internal choice
4. But I did not give a step law for distributing through external choice
5. Question: why is this? It may help to consider the following processes

```
(a -> P) \ {a}
(b -> Q) \ {a}
(a -> P [] b -> Q) \ {a}
```

Table of Contents

Summary

1. In this lecture we have seen how we can reason about process termination
2. In doing so, this has enabled us to differentiate between process termination and deadlock
3. We then went on to consider hiding—effectively the Engineering art of developing black box systems
4. We have seen enough now to move to a key topic—next week we shall study concurrent systems and communication

Summary

1. Advance reading: Schneider, pages 29–41
2. Advance reading: Roscoe, pages 77-86, 55–64
3. Advance reading: Woodcock and Davies, chapter 8